

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Johaan Sam
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 53
CLASS: S3 CSE B	ROLE / FUNCTION: Frontend & State Engine Developer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

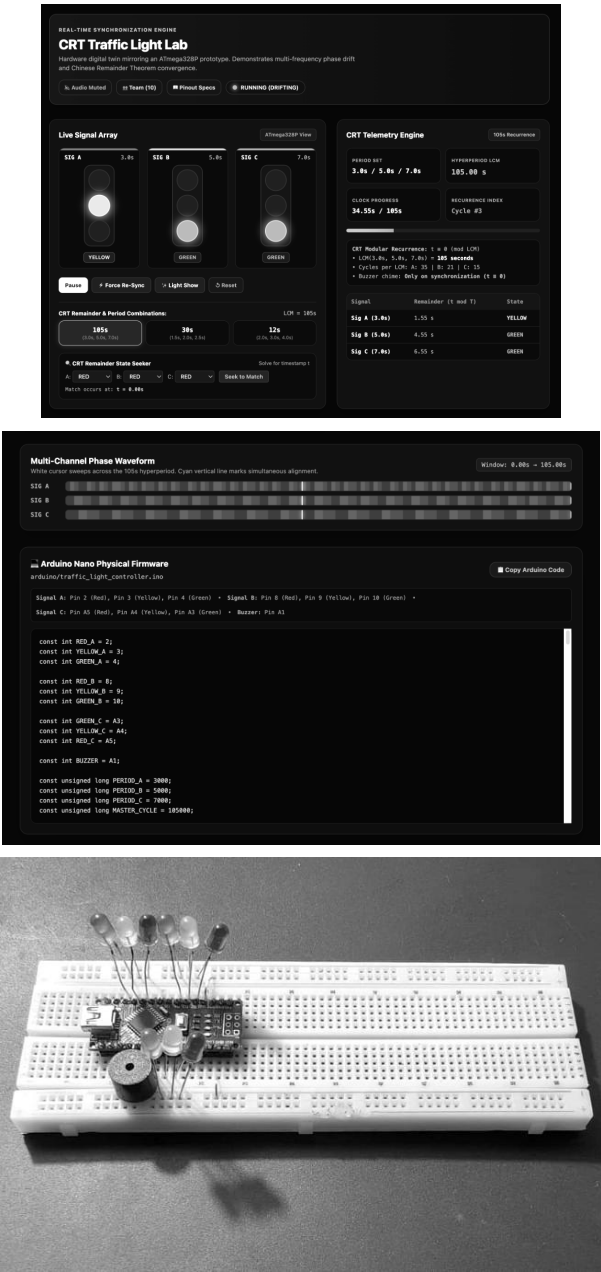
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

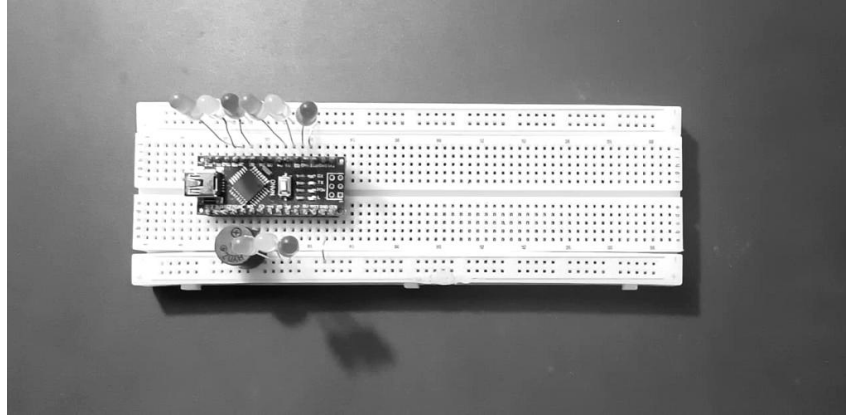
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Johaan Sam

Class: S3 CSE B

Roll No: 53

Role: Frontend & State Engine Developer

Contribution Summary:

I spent over three weeks engineering the interactive web application frontend and real-time state engine using React 19 and TypeScript. Collaborating daily with UI architect Johan Geo and team lead Justin, I developed custom 60 FPS animation `requestAnimationFrame` loops and telemetry sweep cursors. I dedicated significant time to implementing the dynamic CRT remainder solver and wiring real-time state synchronization across interactive traffic monitors, multi-frequency phase waveforms, and acoustic web audio chimes. Through iterative peer code reviews and UI testing sessions, I tuned Framer Motion transitions and component state management for seamless cross-device performance.